Velocity and Acceleration Kinematics Analysis

Robotics Project Milestone 3 · MCTR911 · Team 2

1. Purpose and scope

This report describes the steps taken to derive, implement and validate the velocity and acceleration kinematics of a 6-DOF Lite6 manipulator. The analysis covers the forward problem (joint quantities to end-effector quantities) and the inverse problem (end-effector quantities to joint quantities). Each result is checked two ways: against independent numerical computations, and against the end effector sensed in a MuJoCo simulation of the robot (`lite6.xml`). A ROS2 node then repeats the comparison continuously on live joint-state data.

The report describes the method and the checks as implemented in the code. It does not quote measured error values, because the scripts were not run as part of this write-up. Running them prints the worst-case error for each check.

2. Conventions

- All vectors are expressed in the base frame.
- Twists and accelerations are ordered **[linear; angular]**. MuJoCo returns **[angular; linear]**, so two helper functions (`mujoco_order`, `from_mujoco_order`) swap the halves at the simulator boundary.
- Units are metres, radians and seconds.
- The end effector is the site `attachment_site`. Velocity and acceleration refer to its origin.
- A dot over a symbol is a first time derivative ($\dot{q}$ = joint velocity) and two dots a second derivative ($\ddot{q}$ = joint acceleration).

3. Step 1: Kinematic model (standard DH)

The arm is modelled with standard Denavit-Hartenberg parameters. The transform between consecutive frames is:

$$^{i-1}T_i = Rz(\theta_i) \cdot Tz(d_i) \cdot Tx(a_i) \cdot Rx(\alpha_i)$$

The joint variable enters as $\theta_i = q_i + \text{offset}_i$.

i	θ_i	d_i (m)	a_i (m)	α_i
1	q1	0.2435	0	$-\pi/2$
2	$q2 - \pi/2$	0	0.2002	π

i	θ_i	d_i (m)	a_i (m)	α_i
3	$q3 - \pi/2$	0	0.087	$\pi/2$
4	$q4$	0.22761	0	$\pi/2$
5	$q5$	0	0	$-\pi/2$
6	$q6$	0.0625	0	0

The offsets of $-\pi/2$ on joints 2 and 3 and the zero value of L7 are the choices that make the DH model reproduce the MuJoCo model. The code notes that other offset choices give a pose that does not match the simulator. Chaining the seven transforms (`frames`) gives the pose of every link in the base frame, and the last one is the forward position kinematics (`fk`). Position kinematics is needed here because the Jacobian is built from the frame origins and joint axes.

4. Step 2: Geometric Jacobian

For a revolute chain in standard DH form, joint i rotates about the z-axis of frame $i-1$. Let z_i be that axis and p_i the origin of that frame, both in the base frame, and let p_e be the end-effector position. Column i of the 6×6 Jacobian is:

$$J_i = [\, z_i \times (p_e - p_i) \,;\, z_i \,]$$

The top three rows give the linear-velocity contribution of joint i, and the bottom three rows give its angular-velocity contribution. This is implemented in `jacobian(q)`.

5. Step 3: Velocity kinematics

5.1 Forward velocity

The end-effector twist is linear in the joint velocities:

$$[\, v \,;\, \omega \,] = J(q) \cdot \dot{q}$$

This is `fk_velocity`.

5.2 Inverse velocity

Because the arm has six joints and the Jacobian is square, the inverse is:

$$\dot{q} = J^{-1}(q) \cdot [\, v \,;\, \omega \,]$$

This is `ik_velocity`. Away from singularities it uses a direct linear solve. Near singularities it switches to a damped solve (see Step 5).

6. Step 4: Acceleration kinematics

6.1 Forward acceleration

Differentiating the velocity relation with respect to time gives:

$$[\, a \,;\, \alpha \,] = J(q) \cdot \ddot{q} + J(q, \dot{q}) \cdot \dot{q}$$

This is `fk_acceleration`. The new ingredient is the time derivative of the Jacobian.

6.2 Analytic Jacobian derivative

`jacobian_dot` computes the Jacobian derivative analytically, column by column, with a recursion over the joints:

1. The angular velocity of the preceding link, $\omega_{i-1} = \Sigma(k<i)\ z_k\ \dot{q}_k$, is accumulated as the loop advances.
2. The axis derivative is $\dot{z}_i = \omega_{i-1} \times z_i$.
3. The origin velocity is $\dot{p}_i = \Sigma(k<i)\ z_k \times (p_i - p_k)\ \dot{q}_k$, and the end-effector velocity is $\dot{p}_e = J_lin \cdot \dot{q}$.
4. Applying the product rule to column i gives:

$$J_i = [\, \dot{z}_i \times (p_e - p_i) + z_i \times (\dot{p}_e - \dot{p}_i) \,;\, \dot{z}_i \,]$$

A central-difference version, `jacobian_dot_numeric`, evaluates

$$J \approx [\, J(q + h\cdot\dot{q}) - J(q - h\cdot\dot{q}) \,] \,/\, (2h),\ h = 10^{-6}$$

It exists only as a cross-check of the analytic expression.

6.3 Inverse acceleration

Rearranging the forward relation gives the joint accelerations needed to produce a desired end-effector acceleration:

$$\ddot{q} = J^{-1}(q) \cdot (\, [\, a \,;\, \alpha \,] - J \cdot \dot{q} \,)$$

This is `ik_acceleration`. The velocity-dependent term is subtracted from the commanded acceleration before inverting.

7. Step 5: Singularity handling

The inverse equations are undefined where the Jacobian loses rank. The code detects and handles this as follows.

- **Detection.** `singularity_measure` returns the smallest singular value σ_min, the condition number and |det J|. The arm is treated as near-singular when $\sigma_min < \varepsilon = 0.05$ (`SINGULARITY_EPS`).

- **Damping.** `damped_solve` performs an SVD-based damped least-squares solve. Each singular value s is replaced by the filtered value below, with $\lambda_max = 0.05$.
- **Known singularity.** The wrist is singular when q5 = 0. The self-test demonstrates this with a pose at q5 = 0, checks that σ_min is below 10^{-6}, and confirms that the damped inverse returns finite joint velocities.

The filtered singular value and the adaptive damping term are:

$$s \to s / (s^2 + \lambda^2)$$

$$\lambda^2 = (1 - (\sigma_min / \varepsilon)^2) \cdot \lambda_max^2 \text{ if } \sigma_min < \varepsilon$$

$$\lambda^2 = 0 \text{ otherwise}$$

Away from singularities the solution is therefore exact, and it degrades smoothly as the arm approaches one.

8. Step 6: Supporting position inverse kinematics

`ik_position` is an iterative damped least-squares solver with a step limit of 0.3 rad and joint-limit clipping. Orientation error uses a small-angle cross-product formulation. It is used to find start poses and to include a position round trip in the validation. It is not part of the velocity or acceleration derivation itself.

9. Step 7: Validation

Validation has two stages. Both use randomly sampled joint states: positions at 80% of the joint range, q5 forced to a magnitude between 0.3 and 2.0 rad to stay off the wrist singularity, velocities in $[-1, 1]$ rad/s and accelerations in $[-2, 2]$ rad/s^2.

9.1 Stage A: Self-test (no simulator)

`self_test` runs 200 random samples against checks that do not depend on the DH code.

Check	What is compared	Tolerance
fk_vs_xml	DH forward kinematics against FK built directly from the MJCF body tree (positions and quaternions of lite6.xml)	5×10^{-5}
J_vs_fd	$J\cdot\dot{q}$ against the derivative of FK (finite difference for position, $\dot{R}\cdot R^T$ for angular velocity)	10^{-6}
Jdot_vs_fd	Analytic Jacobian derivative against its numeric version	10^{-6}
acc_vs_fd	Forward acceleration against the derivative of $J(q(t))\cdot\dot{q}(t)$ along a quadratic joint trajectory	10^{-5}

Check	What is compared	Tolerance
vel_roundtrip	ik_velocity(fk_velocity($\dot{q}$)) against $\dot{q}$	10^{-6}
acc_roundtrip	ik_acceleration(fk_acceleration($\ddot{q}$)) against $\ddot{q}$	10^{-5}
ik_position	FK of the IK solution against the target position	10^{-5}

The fk_vs_xml tolerance is looser than the others because lite6.xml stores some quaternions rounded to about six digits (for example 0.707105 instead of 0.7071068). The code comments that this produces roughly 10^{-5} m of difference from the ideal DH model, and that with exact quaternions the two agree to about 10^{-15}.

9.2 Stage B: Equations against MuJoCo-sensed values

`compare_with_mujoco` uses the `MujocoArm` class. The procedure is: set joint angles, velocities and accelerations in the simulator, sense the end effector, and compare with the equations. Gravity is disabled so that sensed accelerations are purely kinematic.

Sensing methods:

- **Position and Jacobian:** site pose from `mj_forward`, and the Jacobian from `mj_jacSite`.
- **Velocity:** `mj_objectVelocity` on the end-effector site.
- **Acceleration (primary):** central difference of the sensed twist along the quadratic trajectory below, with h = 10^{-5}. The code describes this as the most robust way to sense acceleration.
- **Acceleration (optional cross-check):** `mj_objectAcceleration` after `mj_rnePostConstraint`. This is reported but excluded from the overall pass criterion.

$$q(t) = q + \dot{q}\cdot t + \tfrac{1}{2}\cdot\ddot{q}\cdot t^2$$

Checks (all with tolerance 10^{-3} over 100 samples):

1. **Forward:** position, orientation, Jacobian, velocity, and acceleration (finite difference and RNE).
2. **Inverse:** the *sensed* end-effector values are fed into the inverse equations, which should return the original joint quantities. This covers inverse position (judged by the pose MuJoCo senses for the returned solution), inverse velocity (against the known $\dot{q}$) and inverse acceleration (against the known $\ddot{q}$).

Feeding sensed values, rather than values computed from the same equations, makes the inverse check independent of the forward code.

10. Step 8: Online monitoring (ROS2)

`kinematics_monitor.py` repeats the comparison continuously on live data. It is split into a ROS-free core (`MonitorCore`, testable without ROS) and a ROS2 node.

10.1 Processing in MonitorCore

1. Incoming (t, q, $\dot{q}$) samples from `/joint_state` are stored in a buffer of up to 400 samples.
2. The sample spacing is estimated from the last ten samples. Five evenly spaced samples are then picked, counting back from the newest, so that they span a 10 ms window. The window is symmetric about its centre sample, so the fitted slope and the centre values refer to the same instant.
3. Joint acceleration is not published, so it is estimated as the least-squares slope of the published velocities over the window.
4. **Equation side:** FK position, $J \cdot \dot{q}$ and $J \cdot \ddot{q} + \dot{J} \cdot \dot{q}$ are evaluated at the centre sample.
5. **Sensed side:** MuJoCo gives the site position at the centre sample and a twist for each of the five samples. Sensed acceleration is the least-squares slope of those twists.
6. **Inverse check:** the sensed twist and sensed acceleration are passed to the inverse equations and compared with $\dot{q}$ and $\ddot{q}$. If σ_min is below the singularity threshold (for example the all-zero start pose, where q5 = 0), the inverse check is skipped and reported as NaN rather than failing or blowing up.

10.2 ROS2 node behaviour

- The node publishes the equation values, the sensed values and their difference as 15-element arrays on `/kin/equations`, `/kin/sensed` and `/kin/error`. Each array holds position (3), linear velocity (3), angular velocity (3), linear acceleration (3) and angular acceleration (3).
- Each evaluation is classified PASS or FAIL against separate tolerances: position 10^{-3} m, velocity 10^{-3} m/s and acceleration 5×10^{-2} m/s^2. The acceleration tolerance is looser because both the joint acceleration and the sensed acceleration are numerically differentiated from noisy samples.
- About once per second it logs the current errors, the inverse-check errors, σ_min and the worst errors seen so far.
- An optional CSV file records the equation and sensed vectors and the inverse errors for offline analysis.
- Parameters include `model_path` (required), `state_topic`, `rate`, `window`, the three tolerances and `csv_path`.

11. Assumptions and limitations

- Gravity is disabled in the simulator, so the acceleration analysis is kinematic only. Dynamics are not covered.

- The 6×6 Jacobian is square, so the inverse results are unique except at singularities. Joint limits are not enforced in the velocity or acceleration inverse.
- Damping near singularities trades accuracy for finite joint rates. The inverse check in the monitor and the validation are therefore run only away from singular poses.
- The DH model differs from the MuJoCo model by about 10^{-5} m because of rounded quaternions in lite6.xml.
- Online accelerations rely on numerical differentiation over a 10 ms window, which amplifies noise. Their tolerance is correspondingly relaxed.

12. Summary of the analysis workflow

1. Build the DH model and reproduce the MuJoCo pose.
2. Derive the geometric Jacobian from the joint axes and origins.
3. Obtain forward velocity as $J \cdot \dot{q}$ and inverse velocity as the Jacobian inverse applied to the twist.
4. Differentiate to obtain forward acceleration, $J \cdot \ddot{q} + \dot{J} \cdot \dot{q}$, with an analytic $\dot{J}$, and invert it for $\ddot{q}$.
5. Add singularity detection and adaptive damped least squares.
6. Verify each equation against independent numerical derivatives and round trips.
7. Verify forward and inverse results against quantities sensed in MuJoCo.
8. Monitor the same comparison online through a ROS2 node, with per-quantity tolerances and logging.